DEVOPS PROJECT 01

Deploy a Node.js Backend on AWS EC2

Beginner-friendly, step-by-step deployment guide
GitHub • Ubuntu EC2 • PM2 • Nginx

Level	Beginner
Estimated time	7 days · 1 hour per day
Application	Node.js + Express REST API
Cloud service	Amazon EC2
Final result	Public HTTP API served through Nginx

Learning objective: Deploy a working backend to a Linux server, manage it with PM2, route traffic through Nginx, and validate the deployment using practical commands.

Before you begin: This guide creates AWS resources that may incur charges. Review current pricing and free-usage eligibility, set a budget alert, and remove resources when finished.

1. Architecture

Request flow: Browser/client → EC2 public IP on port 80 → Nginx → Node.js/Express on 127.0.0.1:4000.

```
Developer laptop
    |
    | git push
    v
GitHub repository
    |
    | git clone (manual deployment)
    v
AWS EC2: Ubuntu
    |
    +--> Nginx (public port 80)
              |
              +--> Node.js + Express (port 4000, not public)
                        |
                        +--> PM2 manages the process
```

Security group inbound rules: SSH (22) from your IP only; HTTP (80) from anywhere. Do not open port 4000 to the public internet. Add HTTPS (443) later when you configure a domain and TLS certificate.

Tools used

Tool	Purpose
Node.js + Express	Build the HTTP API.
Git + GitHub	Version control and remote source repository.
Amazon EC2 / Ubuntu	Cloud virtual server and Linux environment.
PM2	Manage and restart the Node.js process.
Nginx	Reverse proxy and public HTTP entry point.

2. Seven-day plan

Day	Task	Expected result
1	Build and test API locally	Health endpoint returns JSON
2	Push project to GitHub	Repository contains source code
3	Launch EC2 and connect with SSH	Ubuntu shell access works
4	Install Git, Node.js and Nginx	Server is prepared
5	Clone app and run with PM2	API responds on port 4000 locally
6	Configure Nginx reverse proxy	Public port 80 serves API
7	Validate, document and review	Checklist complete; README ready

3. Day 1 — Build the Node.js API locally

3.1 Check your tools (Windows PowerShell)

```
node --version
npm --version
git --version
```

If a command is not found, install Node.js LTS and Git, reopen the terminal, and retry.

3.2 Create the project

```
mkdir devops-ec2-backend
cd devops-ec2-backend
npm init -y
npm install express
npm pkg set scripts.start="node server.js"
```

3.3 Folder structure

```
devops-ec2-backend/
■■■ server.js
■■■ package.json
■■■ package-lock.json
■■■ .gitignore
```

3.4 Create server.js

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 4000;

app.use(express.json());

app.get("/", (req, res) => {
  res.json({
    message: "DevOps Project 1 - Node.js API",
    status: "running"
  });
});

app.get("/api/health", (req, res) => {
  res.status(200).json({
    status: "healthy",
    service: "devops-ec2-backend"
  });
});

app.get("/api/info", (req, res) => {
  res.json({
    project: "AWS EC2 Backend Deployment",
    environment: process.env.NODE_ENV || "development"
  });
});

app.listen(PORT, "0.0.0.0", () => {
  console.log(`Server listening on port ${PORT}`);
});
```

3.5 Create .gitignore

```
node_modules/
.env
npm-debug.log*
```

3.6 Run and test locally

```
npm start
```

Open these URLs in your browser:

• http://localhost:4000/

• http://localhost:4000/api/health

• http://localhost:4000/api/info

Expected health response:

```
{"status":"healthy","service":"devops-ec2-backend"}
```

Stop the local server with Ctrl+C. Day 1 is complete when all three routes return JSON.

4. Day 2 — Push code to GitHub

Create a new repository at https://github.com/new named **devops-ec2-backend**. For a simple first push, do not initialize the remote repository with a README.

```
git init
git add .
git commit -m "Create Node.js backend API"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/devops-ec2-backend.git
git push -u origin main
```

Replace YOUR_USERNAME with your GitHub username. Verify that server.js, package.json, package-lock.json and .gitignore appear in the repository. Never upload .pem keys, passwords, tokens, or .env files.

5. Day 3 — Launch an EC2 instance

5.1 Create the instance

• Open the Amazon EC2 console: https://console.aws.amazon.com/ec2/

• Select a Region close to you, such as Mumbai (ap-south-1).

• Choose Launch instance.

Setting	Value
Name	devops-project-1
AMI	Ubuntu Server 24.04 LTS
Instance type	Smallest eligible instance suitable for this lab
Key pair	Create and download devops-project-1-key.pem
Storage	Small root volume, e.g. 8 GiB if suitable
Public IP	Enable public IPv4 assignment

5.2 Configure inbound security-group rules

Type	Port	Source
SSH	22	My IP only
HTTP	80	0.0.0.0/0

Do not add port 4000. SSH should not be open to everyone. Keep the downloaded PEM key private and out of GitHub.

5.3 Connect from Windows PowerShell

```
$KEY = "$HOME\Downloads\devops-project-1-key.pem"
ssh -i "$KEY" ubuntu@YOUR_EC2_PUBLIC_IP
```

Replace YOUR_EC2_PUBLIC_IP with the instance's actual public IPv4 address. Confirm the host key when prompted on first connection.

```
whoami
hostname
ls
```

Expected user: ubuntu. Day 3 is complete when SSH opens a shell on EC2.

6. Day 4 — Prepare Ubuntu

Run these commands inside the EC2 SSH session.

6.1 Update packages and install Git, curl and Nginx

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git curl nginx

git --version
nginx -v
curl --version

sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
```

Open http://YOUR_EC2_PUBLIC_IP in your browser. You should see the default Nginx welcome page.

6.2 Install Node.js 22 LTS

```
curl -fsSL https://deb.nodesource.com/setup_22.x -o /tmp/nodesource_setup.sh
sudo -E bash /tmp/nodesource_setup.sh
sudo apt install -y nodejs

node --version
npm --version
```

The Node.js version should report v22.x.x. For production environments, review installation scripts and use approved package sources and version-management practices.

7. Day 5 — Deploy with PM2

7.1 Clone the repository and install dependencies

```
cd ~
git clone https://github.com/YOUR_USERNAME/devops-ec2-backend.git
cd devops-ec2-backend
npm ci
```

Replace YOUR_USERNAME. For a private repository, use a secure authentication method; do not put tokens into the clone URL.

7.2 Install PM2 and start the app

```
sudo npm install -g pm2
PORT=4000 NODE_ENV=production pm2 start server.js --name devops-api

pm2 status
pm2 logs devops-api --lines 20
curl http://127.0.0.1:4000/api/health
```

7.3 Configure PM2 startup

```
pm2 startup
```

PM2 prints a command beginning with sudo. Copy and run the exact command it prints, then save the process list:

```
pm2 save
pm2 status
```

Day 5 is complete when PM2 reports the app as online and curl returns the expected health JSON.

8. Day 6 — Configure Nginx reverse proxy

8.1 Create the site configuration

```
sudo nano /etc/nginx/sites-available/devops-api
```

Paste the following configuration:

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:4000;
```

```
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Save in Nano with Ctrl+O, press Enter, then exit with Ctrl+X.

8.2 Enable the site and validate before reload

```
sudo ln -s /etc/nginx/sites-available/devops-api /etc/nginx/sites-enabled/devops-api
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

Only reload Nginx after nginx -t reports successful configuration. If the symlink already exists, inspect it rather than creating a duplicate.

8.3 Test from your local browser

• http://YOUR_EC2_PUBLIC_IP/

• http://YOUR_EC2_PUBLIC_IP/api/health

• http://YOUR_EC2_PUBLIC_IP/api/info

The requests go to public port 80 and are forwarded internally to port 4000. This lab uses HTTP only; do not send sensitive information. For production, configure a domain and TLS certificate.

9. Day 7 — Validation and troubleshooting

Check	Command (run on EC2)
Node process	pm2 status
Application logs	pm2 logs devops-api
Local backend response	curl -i http://127.0.0.1:4000/api/health
Nginx configuration	sudo nginx -t
Nginx status	sudo systemctl status nginx --no-pager
Nginx error log	sudo tail -n 50 /var/log/nginx/error.log
Listening ports	sudo ss -lntp

Common problems

Problem	What to check
SSH timeout	Public IP, instance state, network path, and SSH source rule.
502 Bad Gateway	PM2 status, app logs, and whether port 4000 is listening.
Port 80 fails	Nginx status, nginx -t, and HTTP security-group rule.
Local works, public fails	Nginx configuration and EC2 security group.
Git clone fails	Repository URL, spelling, and repository access permissions.

10. GitHub README template

Copy the following into README.md in your repository. Replace placeholders and adjust it to match what you actually deployed.

```
# Project 1: Deploy Node.js Backend on AWS EC2

## Overview
A Node.js + Express REST API deployed on Ubuntu EC2.
PM2 manages the Node.js process and Nginx acts as a reverse proxy.

## Architecture
Client -> EC2 port 80 -> Nginx -> Node.js on port 4000

## Tech Stack
- Node.js and Express
- Amazon EC2 (Ubuntu)
- Git and GitHub
- PM2
- Nginx

## API Endpoints
- GET / - service information
- GET /api/health - health status
- GET /api/info - project information

## Run Locally
npm install
npm start

## Deployment Summary
1. Launch Ubuntu EC2 and configure restricted SSH and HTTP.
2. Install Git, Node.js and Nginx.
3. Clone the repository and install dependencies.
4. Start the application using PM2.
5. Configure Nginx as a reverse proxy.
6. Test the public health endpoint.

## Security Notes
- Do not commit credentials, .env files, or PEM keys.
- SSH is restricted to my IP.
- Backend port 4000 is not publicly exposed.
- This learning deployment uses HTTP; production should use HTTPS.

## Troubleshooting
See the project guide for PM2, Nginx, SSH and port checks.
```

11. Interview questions and sample answers

1. What is Amazon EC2?

Amazon EC2 is an AWS service that provides virtual servers in the cloud.

2. Why use Nginx?

Nginx receives client requests and forwards them to the backend as a reverse proxy. It can also handle TLS termination and web-server functions.

3. What is PM2?

PM2 is a Node.js process manager that can restart crashed applications, manage logs and configure startup after a reboot.

4. Why not expose port 4000 publicly?

Clients use the intended entry point, Nginx. Keeping the backend port private reduces unnecessary network exposure.

5. What happens if Node.js crashes?

PM2 can restart the process. I would inspect logs, identify the root cause, and test the health endpoint after recovery.

6. How do you update the app?

Pull the new code, install required dependencies, restart the PM2 process and verify the health endpoint. Later, CI/CD can automate this.

7. What does a security group do?

It acts as a virtual firewall that controls inbound and outbound traffic for associated AWS resources.

8. What is a reverse proxy?

A reverse proxy accepts client requests and forwards them to one or more backend services.

12. Cost control and cleanup

AWS costs vary by Region, instance type, account age, free-plan eligibility, storage, public IPv4, data transfer and other services. A budget alert is a notification, not a hard spending cap. Check current prices before launch.

• Create an AWS Budget with email alerts before deploying.

• Stop EC2 when you are not practising. Stopped instances can still incur EBS storage and some other charges.

• Terminate the instance when the project is complete if you no longer need it.

• Review and remove unused EBS volumes, snapshots, load balancers, NAT Gateways, Elastic IPs and other billable resources.

• Do not assume an EC2 instance or public IPv4 address is free; check the current pricing and eligibility.

Useful official pages:

• EC2 pricing: https://aws.amazon.com/ec2/pricing/on-demand/

• AWS Free Tier: https://aws.amazon.com/free/

• AWS Budgets: https://aws.amazon.com/aws-cost-management/aws-budgets/

Cleanup checklist

• Record any information you need for your portfolio, such as the architecture and deployment steps.

• Terminate EC2 if you no longer need the server.

• Review the EC2 Volumes and Snapshots pages and delete only resources you no longer need.

• Review public IPv4 and networking resources for ongoing charges.

• Confirm that no unintended resources remain in the AWS Billing console.

13. Project completion checklist

• ■ API runs locally and all three routes return JSON.

• ■ Code is committed to GitHub without secrets.

• ■ EC2 is reachable through SSH restricted to my IP.

• ■ Git, Node.js, Nginx and PM2 are installed.

• ■ PM2 manages the backend process and startup is configured.

• ■ Nginx serves the API on port 80.

- ■ Port 4000 is not publicly exposed.
- ■ README includes architecture, commands, endpoints and troubleshooting.
- ■ AWS budget and cleanup have been reviewed.

14. Final result

When the project is complete, you will have deployed a Node.js REST API on AWS EC2, managed its process with PM2, routed public HTTP requests through Nginx, and documented the setup in GitHub.

Recommended next project: Dockerize this same backend and add MongoDB and a frontend using Docker Compose. Reusing the same application helps you explain how deployment evolves from a manual server to automated infrastructure.

Guide prepared for learning and portfolio use. Verify AWS console labels and current pricing before creating resources.